

Software Control System for an Automatic Hair Cutting Machine

Final Report

Student name: _____

Student number: _____

Supervisor(s): _____

School of Engineering, University of Western Australia

Main body word count: 6858

Date of submission: 19 May 2026

STUDENT DECLARATION

My contribution to the project

The report describes the author's personal contribution to the software design, ESP32 bus-servo communication, multi-motor workflow logic, sensor response, pause and recovery mechanism, and preliminary verification of the automatic hair cutting machine prototype.

Use of AI tools

AI tools were used to assist with document formatting, structure, and language polishing. The technical content, project work, design decisions, and reported outcomes remain the author's own work and should be checked before submission.

I certify that the above information is correct, and that the attached work submitted for assessment is my own work and all material drawn from other sources has been acknowledged and referenced.

Student signature _____ Date _____

Supervisor signature _____ Date _____

Project Summary

The project is mainly committed to the design and implementation of automatic hair cutting machines, focusing on solving the problems of insufficient modularity of the original structure and complex multi-motor motion control. The team completed the mechanical structure design and 3D printing prototyping, and my job was to design and implement the bus servo control system based on the ESP32 based on the mechanical platform. The system controls the ST series bus servo via the serial port bus, which supports position mode, wheel mode, multi-motor coordinated operation, motion queue and sensor triggered emergency stop. At present, it has been possible to allow the system to perform haircutting operations according to preset procedures under the condition of ensuring safety. In the future, we need to be able to model the head and trim different hairstyles.

Acknowledgements

Acknowledgements to be completed before submission.

Table of Contents

- 3.1 Introduction
 - 3.1.1 Background and Motivation
 - 3.1.2 Problem Statement
- 3.2 Literature Review
 - 3.2.1 Existing paths to automated haircut scenarios
 - 3.2.2
 - 3.2.3
- 3.3 Project Objectives
- 4. Design Approach
 - 4.1 Design Overview
 - 4.3 Design Constraints
 - 4.4 Alternative Generation Process and Scheme Comparison
 - 4.4.1 Sequence control scheme based on delay().
 - 4.4.2 Scheme of using stepper motor instead of bus servo
 - 4.4.3 Final Option Selection
 - 4.6 Relative multi-turn position control
 - 4.6.1. Control problem description
 - 4.6.2. Lap position feedback and unit position definition
 - 4.6.3. Cross-lap problem analysis
 - 4.6.4. Cross-circle correction method
 - 4.6.5 Calculation of cumulative multi-turn positions
 - 4.6.6. Establishment of preset lap targets
 - 4.6.7. PID control algorithm and speed closed-loop correction
 - 4.6.8. Multi-motor state machine control logic
- 5. Results and Discussion
- 6. Conclusions and Future Work

List of Tables

Table 1: Software verification outcomes.

Nomenclature

Symbol / term	Meaning
ESP32	Microcontroller used for prototype control
M1-M5	Bus servo motors used in the prototype
PID/PD	Proportional-integral-derivative / proportional-derivative control concepts
SC09	Bus servo communication interface
ST3215-HS	Bus servo actuator used in the prototype

3.1 Introduction

3.1.1 Background and Motivation

With the popularization of automation technology and the development of robotics in recent times, more and more traditional service industries that rely heavily on manual experience have begun to try to integrate automation technology into the industry. It is not simply a repetitive process on a stationary workpiece, an automatic hair clipper is a class of human-computer interactive electromechanical systems with obvious engineering challenges. Different from ordinary industrial automation equipment, the working object of the automatic hair cutting machine is the hair near the human head, the equipment has a small operating space, extremely high safety requirements, and the movement trajectory needs to have a certain degree of flexibility and repeatability. An automatic hair cutting machine that can really work requires not only a reasonable mechanical structure, but also a reliable motor drive, stable control logic, timely safety interrupts, and the ability to subsequently integrate with sensors and hair cutting tools. Therefore, even if the project is still in the prototype development phase, the software control system must be scalable and safe from the start, rather than just a single motor rotation demonstration.

For this project, we chose Weixue's ST3215-HS and SC09 bus servos for the actuators, and the controller was ESP32. The reason for choosing a bus servo over a stepper motor is mainly because the bus servo is more in line with the design requirements of automatic hair clippers in terms of integration, ease of control, multi-motor expansion, position feedback, and compactness. The advantages of a bus servo are high integration, simple communication, control of multiple motors via ID, and the ability to read feedback information such as position.

3.1.2 Problem Statement

The core problems that need to be solved at this stage of this project can be summarized as follows: how to establish a bus servo software control framework based on ESP32 in the prototype of the automatic hair clipper, so that it can complete the most basic motor driving tasks, and can support the control of the relative position of the bus servo in multiple turns, the coordination between multiple motors and the sensor-based emergency stop. The question has three levels. The first is the drive level, that is, how the ESP32 communicates stably with the bus servo, and how to complete motor mode switching, position reading, speed control and stop control. The second is the logical level, that is, how to integrate each single motor control command into a scalable motion queue so that multiple actions can be executed sequentially. The third is the safety level, that is, how the program immediately stops the current movement when the sensor or button input changes, rather than waiting for the current action to be executed before responding.

If the software logic is not properly designed, the prototype of the automatic hair cutting machine will have a variety of problems. For example, a wheel mode motor may not stop steadily near the target location or continue to perform the previous queue action after the button is released; Position mode motors and wheel mode motors may be difficult to manage uniformly due to different control methods; If the program relies heavily on the interrupt instruction such as `delay()`, the emergency stop signal may not be read in time; When multiple motors share a serial port bus, if the read and write order is confused, it may also cause control delays or state judgment errors. These mistakes can eventually lead to huge personal safety issues. Therefore, the software design goal of this project is not to simply make the motor rotate, but to establish a control framework with a clear structure, debuggability, scalability and safe interruption capability, which can lay a solid foundation for future research while meeting current needs.

3.2 Literature Review

3.2.1 Existing paths to automated haircut scenarios

At present, the research and practice in the field of automatic haircutting mainly focuses on the use of multi-degree-of-freedom robotic arms as the core components of haircutting, and the relevant literature on helmet-type automatic haircutting devices has not been consulted. The idea of the mainstream solution is to hold the haircutting tool at the end of the robotic arm, identify the user's head contour and hairstyle through the machine vision system, and then plan the path to complete the haircutting task. At present, there are intelligent automated multi arm unmanned ground vehicles that provide hairdressing services to people, with 29 degrees of freedom. [8] According to Li, it provides a set of automatic haircutting system design schemes based on six-degree-of-freedom robotic arms, in which three degrees of freedom are used for position control and the other three degrees of freedom are used for attitude control such as pitch, yaw and roll. [1] Even in some cases more redundancy is required. However, under the premise of ensuring high precision, increasing speed and protecting head safety has become a problem. Zheng, K and the team point out that as the degrees of freedom increase, the number of terms included in the kinetic equation increases geometrically. [2] Therefore, through these literatures, we can see that the existing scheme has strong limitations: multi-axis robotic arms will lead to high technical difficulties, and at the same time, it is difficult to achieve accurate and efficient control, which will also lead to safety risks. These factors are not conducive to the development and promotion of automatic hairdressing devices.

3.2.2

PID control is one of the most common and fundamental control methods in robotics and electromechanical systems. Lopez-Sanchez and Moreno-Valenzuela point out that PID control is widely used in industrial and commercial electromechanical products due to its simple structure and ease of implementation; Its three parts, proportion, integral and differentiation, correspond to the current error, past error and change trend of the system, respectively, so it has good engineering intuitiveness. The literature also mentions that PID can reduce steady-state errors, compensate for partial perturbations, and increase damping effects for closed-loop systems. Although the research object is a quadcopter UAV, the view that PID is easy to implement and has strong practicality in electromechanical systems has reference significance for the bus servo control of this project. [] Similarly, Rahimi Nohooji et al. pointed out that PID control is still widely used for practical engineering control problems due to its simple structure, intuitive concept, and low cost of implementation; At the same time, the choice of PID parameters will directly affect the performance of the closed-loop system. This literature further illustrates that complex adaptive or reinforcement learning methods, while optimizing PID parameters, can also increase system complexity and computational burden. []

Therefore, this project borrows the ideas of "error drive" and "damping suppression" in PID/PD control, but simplifies it according to the characteristics of the bus servo wheel mode. Through single-turn position feedback, cross-lap judgment, cumulative position and speed feedback control, the system can realize relatively multi-turn motion control in wheel mode, providing the basis for the five-motor workflow.

3.2.3

Singh et al.'s study discusses robotic kinematics modeling, inverse kinematics solving, and real-time pick-and-place applications with 5-degree-of-freedom tandem robotic arms. The literature points out that this type of robotic arm can be used for industrial applications such as grasping and sorting, and uses a serial bus servo as a joint actuator. An important feature of the Serial Bus Servo compared to ordinary servos is that multiple motors can be connected in series and drive different joint actions through control commands. The literature also states

that the Serial Bus Servo has built-in position feedback and voltage feedback. This is important for multi-degree-of-freedom robotic systems, as the controller can not only send motion commands to the servo, but also read the actuator status to determine the position of the joints and the operation of the system. For the automatic hair clippers in this project, the M1 to M5 also form a multi-actuator system, where motor feedback is the basis for multi-motor coordination, position judgment, anomaly detection and debugging output. This is similar to the software control idea of this project, and the feedback capabilities of the serial bus servo and the multi-motor connection make it more suitable for this multi-motor prototype system than a normal PWM servo.

3.3 Project Objectives

Based on the above questions, in this project, I have set the following goals for the software part I am responsible for. First, complete the communication between the ESP32 and the bus servo to ensure that the ESP32 development board can accurately identify and change the servo ID, read position feedback and send motion instructions. Second, it realizes the switching between position mode and wheel mode and perfectly realizes the basic drive of these two modes, so that the system can choose single-turn positioning or continuous rotation according to different mechanism needs. Third, the relatively multi-turn control logic is designed, and the wheel mode motor can move relative according to the target number of turns through single-turn position feedback, cross-lap judgment and cumulative position calculation. Fourth, establish motion queues and multi-motor coordination logic to enable multiple actions to be performed continuously, and also enable motors of different modes to run under the same software framework. Fifth, safety and reliability measures are added, including pressure sensor triggering, button pause recovery, current protection, stall detection, and action timeout protection.

4. Design Approach

4.1 Design Overview

The personal contributions of this project mainly focus on the software development, operation process design, multi-motor collaborative logic, sensor response, pause and recovery mechanism and basic safety protection logic of the motor control system. The goal of the software system is not to control a single motor individually, but to allow multiple actuators to form a complete action flow according to the working needs of the automatic hair clipper. Therefore, the project process will focus on how to control multiple motors stably, safely, and repeatably to perform continuous motion.

The software control system of this project includes a total of 5 motors, which are represented by M1-M 5, as shown in the figure below.

The M1motor controls the rotation of the base, the M2motor moves longitudinally, the M3motor controls the distance between the scissors and the head, the M4motor moves the scissors left and right to ensure that the scissors can cut the hair, and the M5motor controls the opening and closing of the scissors.

To describe the spatial relationship between the five motors, the head can be approximated as the surface of a sphere. M1 controls latitude, M2 controls longitude, M3 controls altitude, and M4 and M5 control precisely. The operation logic that needs to be completed for the project is to keep the M1motor still and the M2 motor to rotate 1/20 turn; The M3 motor drives the scissors to move slowly in the direction of the head, and when the pressure sensor is under pressure, it stops immediately, and then rotates back 1/10 turn; The M4motor then drives the scissor module to move parallel away from the scalp part that needs to be trimmed, and then back to capture the hair. Finally, the M5 motor controls the scissors to cut the hair strands, and then return to the position; Then the M3 motor returns to the initial position. Repeat this action until the M2 motor rotates 3 turns in total, at this time

the M1 motor rotates counterclockwise 1/4 turn, and then repeats the above operation until the M1 motor rotates a total of 7 turns. At this point, all movements are over, and the collection of all movements can completely cover the contour of the head and complete the haircut needs. At the same time, the software system has current detection, which can shut down the system at any time when the current is large; With a start-stop button, the ability to manually control the operation of the system is a manual safety measure.

4.3 Design Constraints

The software design of this project is constrained by mechanical space, main control chip type, power supply capacity, communication mode, manufacturing method and security requirements.

First, in terms of space constraints, the motor installation position, movement direction, and mechanical stroke are limited by the actual structure. Then the software design must adapt to the existing mechanical structure, and we cannot assume that the motor can move infinitely. For example, M4 and M5 need to be switched between specified angles or positions, not beyond the mechanical allowable range. In this mode, the position information of the bus servo is still retained after the power is lost, which means that the angular position of the motor is absolute, so the zero point needs to be reset during the configuration to avoid the rotation angle exceeding the limit.

Second, in terms of materials and manufacturing methods, structural stiffness, connection strength, and impact resistance are limited due to the 3D printing of mechanical structures. Therefore, when the software controls the motor to rotate, it cannot use too high speed or excessive impact. So, in the software part of the project, I have limited the motor speed, acceleration, action timeout, and stuck detection in the program.

Then, in terms of cost and hardware resources, the system employs ESP32 and bus servos as the primary hardware platform. The ESP32 is low-cost, small, and multitasking, making it suitable for prototyping system development. However, the ESP32 shares the same serial bus with multiple servos, so the program needs to control the communication frequency to avoid communication delays or data loss caused by frequent reads and writes. In the code, the motor control task runs every 20 ms, the sensor task detects every 5 ms, the button task detects every 10 ms, and the debug task outputs status information every 1000 ms to balance real-time and communication burden.

Finally, in terms of safety constraints, since automatic hair cutting machines are a type of equipment that runs close to the human head, stop mechanisms and abnormal state protection must be considered even if the current system is still in the prototype stage. Therefore, we have added functions such as press button pause/resume, pressure sensor trigger stop, global current protection, single motor current protection, stall protection and action timeout protection in the software. When the system detects abnormal current, jamming or timeout, it will enter the emergency stop state we define, and call `stopAllMotors()` to stop all motors to prevent the action from continuing to perform and ensure personal safety.

4.4 Alternative Generation Process and Scheme Comparison

Before finalizing the final control option, the project considered two alternative ideas: `delay()`-based sequential control and the use of stepper motors instead of bus servos. Both schemes are intuitive and suitable for early functional verification, but there are certain limitations when combined with the automatic operation and safety requirements of the five motors.

4.4.1 Sequence control scheme based on delay().

In the early days of debugging, the most straightforward approach was to have the program perform actions in a fixed order, such as letting M2 move for a while, then letting M3 move forward, then controlling M4 and M5 to reach the specified position, and finally letting M1 step forward. Wait for the current action to complete by delay() between each step. This method is simple and intuitive, suitable for testing whether a single motor can operate normally, and also suitable for verifying a basic operation.

But when the system expanded to five motors, the problem with delay() became apparent. It will block program operation and the ESP32 will not be able to handle button inputs, pressure sensor signals, motor current feedback, and stall detection in time while waiting. If the M3 is approaching and the pressure sensor has been triggered during delay(), the program may not stop the motor immediately, which is risky for automatic hair clippers that operate close to the human body. In addition, delay()-based methods essentially estimate whether an action is completed in terms of time, rather than based on actual feedback, meaning that when the time for delay() is up, the program automatically runs to the next step. However, the actual movement of the motor will be affected by the load, power supply voltage and mechanical resistance, and the fixed time does not guarantee that the motor will reach the target position. When the load is large, the motor may not be in place, but the program has already entered the next step; With a small load, the motor may have already completed its operation, but the program is still waiting. This reduces control accuracy and repeatability, and is not conducive to the implementation of pause and resume functions. Therefore, the delay() scheme is only suitable for early testing and not as the final running logic.

4.4.2 Scheme of using stepper motor instead of bus servo

The use of stepper motors was also considered at the beginning of the project. The stepper motor can control the rotation angle by the number of pulses, as long as the step angle and subdivision are known, the number of pulses corresponding to one turn can be calculated, so as to achieve the movement of one, two or more turns. Therefore, from the perspective of multi-turn motion, the stepper motor has certain advantages and is not limited by the 0–360° range of the normal servo position mode.

However, most ordinary stepper motors belong to open-loop control. The controller sends as many pulses as the motor completes by default, but if the load is too large, the structure is stuck, or the acceleration settings are not suitable, the motor may lose steps. Once a step is lost, the program cannot know that the actual location has deviated from the target. For ordinary mechanical platforms, this may just be a positioning error; But for automatic hair cutting machines, which operate close to the human body, the lack of feedback poses a greater safety risk. To solve this problem, you will also need to add encoders, limit switches, or closed-loop stepper drives, which will increase system complexity and cost.

Wiring and hardware integration for stepper motors are also more complex. Each stepper motor usually requires signal lines such as STEP, DIR, EN, etc., as well as a separate driver. If the system contains five motors, multiple drives, power distribution, and more control lines are required. In contrast, a bus servo can share a serial bus with multiple motors, distinguishing between different motors by ID, resulting in simpler wiring and better suited to current prototype systems.

The bus servo can also provide direct position, current, and speed feedback, which is used in this project for zeroing judgment, current protection, stall protection, and serial port commissioning. For example, when M2 returns to zero, the program can judge whether it is close to the bottom based on current, load, and speed changes. During operation, it is also possible to judge whether the motor is abnormal through feedback. Therefore, although the stepper motor is more direct in multi-turn control, it is more reasonable to continue to

use the bus servo considering the feedback capability, wiring complexity, safety protection and current mechanical structure. This project finally realizes the relative multi-turn control of the bus servo through wheel mode, single-turn feedback reading, cross-lap judgment and cumulative position calculation.

4.4.3 Final Option Selection

After comprehensive comparison, although the sequence control based on `delay()` is simple, it is not real-time enough, and it is difficult to respond to pressure sensors, buttons, and abnormal protection in time. Stepper motors, while suitable for multi-turn motion, have the risk of step loss and increase the complexity of the drive, wiring, and mechanical fitting. Therefore, the project finally adopted the ESP32 + bus servo + non-blocking state machine control scheme. This solution can continuously read feedback and external inputs during motor operation, and supports pause, resume, current protection, stall protection, timeout protection and emergency stop logic, which is more in line with the actual needs of automatic hair clipper prototypes.

4.6 Relative multi-turn position control

In this project, some bus servos need to complete continuous rotation beyond the range of single turns and need to stop steadily around a preset number of turns. The position mode of ordinary bus servo can realize closed-loop positioning in a single turn, but it is difficult to directly complete the position control of multiple turns. Although the wheel mode can rotate continuously, it usually only receives speed commands and cannot directly input the target position for more than one turn. To solve this problem, this project proposes a relatively multi-turn control method based on single-lap position feedback, cross-lap judgment and cumulative position calculation. This method extends the servo's original 0–4095 turn feedback to a continuous cumulative position and establishes a relative target displacement based on the current position, allowing the wheel mode motor to operate according to a preset number of turns.

4.6.1. Control problem description

The ST3215-HS bus servo has two main modes of operation: position mode and wheel mode. In position mode, the servo can perform closed-loop positioning based on the target position, but its position feedback is limited to a single lap range. The bus servo used in this project corresponds to 4096 position units per turn, so the position feedback range for a single turn can be expressed as:

$$p \in [0, 4095]$$

That is, divide a circle into 4096 parts. Since the magnetic encoder can only display values in this range, it cannot exceed this range even when switched to wheel mode. This range is fairly accurate in describing the angular position of the motor in one turn, but it does not directly indicate how many full turns the motor has turned. For example, when the motor has been rotated three full turns, the servo feedback value will still only be the value before the rotation, and will not directly show the information that it has been rotated three times. Therefore, although the bus servo can control the position of the servo in a single turn range, it cannot directly give a multi-turn target such as 3 turns or -2 turns.

Switching to wheel mode solves the problem of continuous rotation. In wheel mode, the servo is no longer confined to a single turn angle range, but can continue to rotate forward or reverse like a regular DC motor. However, in wheel mode, the controller usually sends speed instructions instead of target position commands. That is, the program can command the motor to run at a certain speed, but without additional feedback processing, the program does not know how many revolutions the motor has turned or whether it has reached a preset number of turns.

Therefore, the core problem to be solved in this project can be summarized as follows: how to use the single turn position feedback of the servo itself in wheel mode to calculate the relative number of turns that the motor has turned, and stop when it reaches the preset number of turns. In order to achieve this goal, this project adopts the method of "single lap position feedback + cross-lap judgment + cumulative position + target error judgment". This method does not change the servo hardware structure and does not need to add an additional encoder, but extends the single-turn feedback to a continuous relatively multi-turn position through a software algorithm.

4.6.2. Lap position feedback and unit position definition

In the control program of this project, a full turn of the bus servo is defined as 4096 unit positions. For the sake of subsequent derivation, the corresponding position count of a circle can be directly defined as:

where represents the number of unit positions corresponding to the rotation of the motor for a full turn. If the feedback value of the servo's rotation position during the k th sample is , then :

What is here is not a continuous multi-turn position, but the encoded value of the current axis position of the motor within a single turn range. Since a full circle corresponds to 360° , the conversion relationship between the position unit p and the angle θ is:

For an arbitrary increment ΔP , its corresponding number of turns R can be expressed as:

$$R =$$

Conversely, if you want the motor to turn the R turn, the corresponding position increment is:

For example, when the target displacement is 12288 unit positions, the corresponding number of turns is:

This means that the motor needs to turn forward 3 times. If the target displacement is -8192 unit positions, then:

This means that the motor needs to be turned in reverse 2 turns. Therefore, in software control, as long as it is possible to establish a continuous cumulative position, the position unit can be used instead of the number of laps for calculation. This representation is more suitable for program implementation than direct use of angles, as the servo feedback itself is given in integer position units of 0–4095.

4.6.3. Cross-lap problem analysis

In wheel mode, the motor can rotate continuously, but the feedback position is always limited to between 0 and 4095. When the motor turns over the single turn boundary, the feedback value will jump, which is the most critical problem in cumulative multi-turn control.

Taking forward rotation as an example, when the motor is close to the end of a turn, the actual motor position may change as follows: starting from the 4090 position, passing through the 4095 position, then passing through the 0 position, and finally reaching 5 location. At this time, the feedback value may jump directly from 4090 to 5

Then when the feedback value jumps from 4090 to 5 , you get:

Numerically, this seems to indicate that the motor rotates 40 8 5 units in the opposite direction in a sampling cycle, but in reality, the actual displacement should be close to +11 unit. Therefore, direct calculation of the feedback difference results in displacement estimates that are incorrect in both direction and magnitude at the cross-lap position. Similarly, a similar problem occurs when reverse rotation. This shows that without cross-lap correction, the cumulative position will have a huge error every time it crosses the 0/4095 boundary. As the motor runs for more time, this error accumulates and eventually the program cannot determine the number of turns the motor has actually turned or stop at the preset number of turns. Therefore, wheel mode multi-turn

control must solve two problems: first, identify whether the current sampling cycle has crossed; Second, if a cross-lap occurs, the anomaly difference during cross-lap is corrected to the true relative displacement increment.

4.6.4. Cross-circle correction method

In order to determine whether two adjacent position readings have cross-turned, the half-turn threshold is used as the basis for judgment, because due to the physical properties of the motor, it is difficult for the motor to rotate more than half a turn in a detection cycle. Since a circle is 4096 unit positions, a half circle is:

Under normal control cycles and limited speeds, the motor should not turn more than half a turn between two adjacent sampling cycles. Therefore, if the raw difference is greater than 2048 or less than -2048, then we can assume that this is not a true displacement from ordinary rotation, but rather a jump caused by the feedback value crossing the 0/4095 boundary. In this case, the original position difference is first calculated:

Then correct according to the half-turn threshold:

When it is , it means that the current feedback value suddenly becomes larger, usually corresponding to the motor crossing from near the 0 position to the 4095 position in reverse. The direct difference is a large positive number, but the true displacement should be a small displacement in the negative direction, so one turn needs to be subtracted, that is:

When , it means that the current feedback value suddenly becomes smaller, usually corresponding to the motor from near the 4095 position to near the 0 position. The direct difference is a large negative number, but the true displacement should be a small displacement in the positive direction, so a circle needs to be added:

When the difference does not exceed the half-circle threshold, it means that no cross-lap has occurred, and the original difference can be used directly.

With this correction method, the jump of the feedback value at the 0/4095 boundary can be converted into small increments in real continuous motion. For example, if the feedback value changes from 4095 to 0, the original difference is -4095, less than -2048, so it is corrected to:

This is consistent with the true displacement when crossing the circle in the forward direction. If the feedback value changes from 0 to 4095, the original difference is 4095, which is greater than 2048, so it is corrected to:

This is consistent with the true displacement when crossing the lap in reverse.

4.6.5 Calculation of cumulative multi-turn positions

After completing the cross-circle correction, each control cycle can obtain a relative displacement increment that is closer to the real motion. The program then continuously accumulates the displacement increment to obtain the cumulative position of the motor from the initial position or the calibration position. The formula for calculating cumulative positions is:

Where, represents the cumulative position at the kth sampling, represents the cumulative position at the previous sampling time, and represents the increment of the latest period after cross-circle correction. When we initialize or calibrate the system, we set the initial value of the cumulative position to:

After that, as the motor rotates continuously in wheel mode, the cumulative position is no longer limited by the 0–4095 lap range, but can be continuously increased or decreased. For example, if the motor rotates one turn forward from the initial 0 position, the cumulative position is about:

If the motor rotates forward three times, the cumulative position is about:

If the motor rotates two more times in reverse, the cumulative position is about:

Once this is done, although the servo hardware feedback is still only a single turn position, the cumulative position in the software already has the ability to display a relatively multi-turn displacement.

4.6.6. Establishment of preset lap targets

After obtaining the cumulative multi-turn position, the preset lap control in wheel mode can be converted into relative target position control. Suppose you want the motor to turn the R-turn from the current position, the target relative displacement is:

If the current cumulative position is , the target cumulative position can be expressed as:

The formula indicates that the target position is not a fixed value within the range of a servo lap, but a relative displacement is added to the current cumulative position. This way, even if the motor has turned multiple times, the next action can still continue to build a new target from the current position. For example, when the current cumulative position of the motor is , and you want the motor to rotate 3 revolutions in the forward direction:

If you want the motor to turn 2 turns in reverse, then:

Since the target position is based on the current cumulative position, this method supports continuous relative action. For example, a motor can perform +3 turns first, then -2 turns, and then +0.25 turns. Each action only needs to recalculate the P_target based on the current cumulative position and does not need to rely on the servo turn position pattern. At the same time, in the actual control process, the program will calculate the error between the current position and the target position in each control cycle, that is, how far is still from the target position:

When the error is large, the wheel mode motor continues to move; When the error gradually decreases and the stop conditions are met, the program sends a zero speed command to the motor to stop.

4.6.7. PID control algorithm and speed closed-loop correction

After establishing the cumulative position and the target position, the position closed-loop can theoretically be controlled by standard PID:

where is the output control quantity. For this project, the program does not use the I term because the integration term tends to generate integration accumulation when the motor is blocked or approaches the target, but adopts a simplified PID form consisting of proportional error and speed feedback. The motor speed is obtained by the cumulative position difference:

When the target position remains constant for an action cycle, there are:

Therefore, the differential term in a standard PID can be written as a damping term opposite to the velocity. let represents the velocity feedback coefficient, then the control law actually used in this project is:

In this formula, the speed of movement is determined according to the distance from the target; When the error is large, the motor speed is higher, and when the error is reduced, the motor naturally slows down. It is used to suppress overshoot when approaching the target quickly, which is equivalent to adding damping to the system. This form is more suitable for the preset number of turns stop control of the wheel mode motor than the proportional control alone.

4.6.8. Multi-motor state machine control logic

The core idea of the state machine is that the system is only in a clear state at each moment, and the program determines the next step based on the current state, sensor input, motor feedback and protection conditions. This allows for continuous inspection of pressure sensors, buttons, current, speed, and timeout conditions during motor movement. For system-level status, we represent it as:

It means waiting for starting, M2 returns to zero, indicates initial preparation, indicates the execution of a local shear cycle, means switching the circumferential area after completing a layer, indicates manual pause, indicates the end of all actions, and indicates abnormal emergency stop. The local action is controlled by the action-level state, which can be expressed as:

So based on these, our system state transition can be abstracted as:

It contains the current motor cumulative position, motor speed, current feedback, pressure sensor status, button status, and action running time. State transfer does not rely on fixed waiting times, but on the conditions we set. For example, the transfer of the M3 proximity state can be simplified to:

This rule states: when the pressure sensor is triggered, the M3 immediately stops approaching and enters a retreat; If the pressure is not triggered but the travel or time exceeds the limit, the system enters an emergency stop; Otherwise the M3 continues to approach slowly. Security protection participates in state transfer as the highest priority. You can write an exception condition as:

where it means overcurrent, indicates blockage, indicates action timeout, and indicates manual stop or pause request. When E is true, the system no longer continues the current workflow, but instead executes the stop logic first:

Through this state machine structure, the complex five-motor action is split into checkable, pausable, and resumable stages. Its advantage lies not in the reduction in the number of actions, but in the fact that each stage has clear entry conditions, completion conditions and abnormal exit conditions. As a result, the system is able to complete the preset haircut path while preserving sensor response and safety protection capabilities.

5. Results and Discussion

Table 1: Software verification outcomes.

Test item	Expected result	Observed result	Status
Servo communication	M1–M5 feedback readable	The serial port successfully reads position/current/speed	Pass
mode switching	The bus servo is capable of switching between position mode and wheel mode	Ability to switch motor modes according to project requirements	Pass
Multi-turn control	The servo is capable of operating according to predetermined position commands for more than one turn	Under motion commands of more than one turn, the motor runs well	Pass

Workflow	The multi-motor operates in a set order	Ability to perform in a preset sequence of actions	Pass
Security measures	The system stops suddenly when encountering a safety problem	The system can be stopped immediately in case of blockage, timeout, or overcurrent	Pass

Based on the current code, the software results I have completed can be roughly summarized in five aspects. First, the project completes the integrated control of the ESP32 with five bus servos. It is mainly reflected in the fact that the program can read the ST servo position from M1 to M4 and the SC09 position of M5 at startup, and control different types of servos through a unified workflow. Second, it realizes relatively multi-turn control in M1, M2, and M3 wheel modes. The cross-lap judgment program converts the single-turn feedback into continuous displacement, and the speed is controlled by Kp and Kv. Third, the M2 zero return and logical zero point reset are realized, so that the vertical direction action does not rely on manual placement to the precise position in advance. Fourth, an automatic working cycle composed of multiple action states is realized, so that the five motors can complete the hierarchical action in sequence. Fifth, the necessary safety and debugging mechanisms for the prototype stage are implemented, including button pause recovery, pressure trigger stop, current protection, stall protection, timeout protection, and detailed serial port output.

These results show that the current software has evolved from a single servo test program to a more complete multi-motor workflow control program. In particular, the state machine structure makes the program more scalable. If you need to modify the M4 action sequence, increase the number of M5 actions, or change the M1/M2 step length, you only need to adjust the parameters or state transitions without rewriting the entire program. However, there are still some limitations to the current design. First of all, the system is mainly verifying the motion logic, and the shearing effect has not yet been verified in the real barbering environment. The motion of the M4 and M5 simulates hair capture and cutting, but the softness, density, orientation, and length of the actual hair can affect the cutting result. Secondly, M3 relies on pressure sensors to determine contact, but a single pressure signal cannot fully reflect the spatial relationship between scissors and head, and it will still need to be combined with distance sensors, limit structures, or visual detection to improve safety in the future. Thirdly, the multi-turn position in wheel mode is a relative cumulative position, and the absolute multi-turn position cannot be retained after the system is powered off, so it needs to be recalibrated every time it starts. Finally, the current software emergency stop cannot replace the hardware emergency stop and mechanical protection structure, and if it is used for real human testing in the future, it must also add independent hardware emergency stop, mechanical limit, protective shield and stricter safety verification. For the completion of the project objectives, I summarize as described in the following table:

6. Conclusions and Future Work

This project completed the software design and preliminary verification of the control system of the five motors in the prototype of the automatic hair cutting machine. The main achievement of the project is to realize the multi-motor cooperative operation and relatively multi-turn control in wheel mode, which makes up for the limitation of single-turn positioning in normal position mode without adding an external encoder. In terms of safety control, the system adds pressure sensors, current detection, stall protection, timeout protection and manual start-stop buttons. Compared with the earlier delay()-based sequential control, the state machine scheme used now has better real-time responsiveness and is more suitable for subsequent function expansion.

Overall, the project has completed the basic objectives of the software control part of the automatic hair clipper, proving that the ESP32 and the bus servo can form a functionable multi-motor prototype control system. The current results are still in the functional verification stage and cannot be directly used for real human haircuts, but they have provided a basis for subsequent shear effect testing, path optimization and safety system improvement.

Future work should prioritize verifying the true shear effect. The current system mainly proves that the movement process is feasible, but the actual effect of M4 capturing hair and M5 cutting hair still needs to be tested by wig models. The shearing success rate at different hair lengths, densities, and directions should be recorded, and the M3 approach distance, M4 lateral movement, and M5 closure timing should be adjusted accordingly. At the same time, the proximity detection of the M3 needs to be further enhanced. Current systems mainly rely on pressure sensors to determine the contact status, but a single pressure signal is easily affected by the installation position and contact angle. Subsequently, distance sensors, limit switches, or visual detection can be added to enable the system to obtain both distance and pressure information, thereby improving the reliability of proximity actions. Finally, security systems need to expand from software protection to hardware protection. If the system enters the real human testing stage in the future, independent hardware emergency stop, mechanical limit, anti-pinch structure, protective cover and power-level protection should be added to ensure that dangerous actions can still be stopped in the event of software failure or abnormal communication.

References

Reference list to be completed before submission. Ensure all literature, standards, datasets, figures, tables, and copied or adapted technical material cited in the report are included here in a consistent referencing style.

Appendices

Appendices may include source code extracts, raw test logs, wiring diagrams, and additional verification data.